PROJET BTS SIO – 1re Année

SCRIPTING BASH – AUTOMATISATION ET ADMINISTRATION LINUX

Bryan Floranceau

1. CONTEXTE ET BESOIN

Ce TP porte sur l'apprentissage du scripting Bash sous Linux. L'objectif est de comprendre le fonctionnement des commandes, des scripts et des structures de contrôle afin d'automatiser des tâches système courantes. Le scripting Bash est essentiel pour l'administration système et permet de gagner du temps et de fiabiliser les opérations.

2. OBJECTIFS

• Comprendre les bases du langage Bash

• Manipuler les variables et les entrées utilisateur

• Utiliser les structures conditionnelles

• Mettre en place des boucles

• Gérer les fichiers et permissions

• Créer des scripts automatisés

3. COMMANDES PRINCIPALES

echo : afficher du texte
cat : afficher le contenu d'un fichier
read : récupérer une saisie utilisateur
tr : transformer des caractères (ex: majuscule → minuscule)
sed : modifier du texte avec expressions régulières
chmod : modifier les droits (exécution, lecture, écriture)
ls / ll : afficher les fichiers
seq : générer une suite de nombres

4. VARIABLES ET ARGUMENTS

$variable : accéder à une variable
$? : code retour de la dernière commande (0 = succès)
$$: PID du script
$! : PID du dernier processus
$0 : nom du script

$1, $2... : arguments du script
$@ : tous les arguments

5. TESTS ET CONDITIONS

-f : vérifier si fichier existe
-d : vérifier si dossier existe
-e : vérifier existence
-r(lecture) / -w(écriture) / -x(exécution) : droits

Comparateurs :
-eq (égal), -ne (différent), -gt (plus grand), -lt (plus petit), -ge, -le

6. STRUCTURES CONDITIONNELLES

```
if [ condition ]; then
   commande
elif [ condition ]; then
   commande
else
   commande
fi

case variable in
   valeur) commande ;;
esac
```

7. BOUCLES

while : exécute tant que condition vraie
for : boucle sur liste ou nombre

break : arrêter la boucle
continue : passer à l'itération suivante

8. EXEMPLE DE SCRIPT – CRÉATION FICHIER

```
#!/bin/bash
fichier="message"
if [ -f "$fichier" ]; then
   echo "I love Bash" >> $fichier
else
   echo "I love Bash" > $fichier
fi
```

9. EXEMPLE DE SCRIPT – BOUCLE

```
#!/bin/bash
for i in $(seq 1 5)
do
   echo "Hello $i"
done
```

10. Preuves (Capture)

1)

```
#!/bin/bash
#
if[ -f message -a -w  message ]
then
        echo "I love Bash" >> message
elif [ ! -e message ]
        echo "I love Bash" > message
elif [ -f message -a ! -w message ]
       chmod u+w message
       echo "I love Bash" >> message
else
       echo "message existe mais n'est pas un fichier ordinaire" >&2
fi
~
```

2)

```
#!/bin/bash

while test
do
      commande
done
~
```

3)

```
#!/bin/bash
#
if [ $# -eq 1 ]
then
        echo "$1 ? "
        read rep
        case $rep in
                [oO] | [yY] | [oO][uU][iI])
                        echo "réponse oui"
                        ;;
                [nN] | [nN][oO][nN])
                        echo "tres bien, je respecte votre choix"
                        ;;
                *)
                echo "Desole, je ne comprends pas votre reponse"
                ;;
        esac
fi
```

4)

```
#!/bin/bash
MOT=${1:?"Vous devez indiquer un mot"}
NB=${2:-5}
i=$NB
while [ $NB -gt 0 ]
do
      echo "$MOT"
      i=$(( $i - 1 ))

done
```

11. BONNES PRATIQUES

#!/bin/bash en début de script
chmod +x script.sh pour exécution
bash -n script.sh pour vérifier syntaxe
Utiliser des variables protégées avec ""
Commenter le code

12. CONCLUSION

Ce TP m'a permis de comprendre et maîtriser les bases du scripting Bash. J'ai appris à utiliser les commandes Linux, les structures conditionnelles et les boucles pour automatiser des tâches. Ces compétences sont essentielles pour l'administration système et la gestion de serveurs Linux.